Итоговый тест

В этой главе мы рассмотрели много материала. Если вы дошли до этого момента, то я вас поздравляю — вы проделали немало работы и это уже хороший шаг на пути к изучению языка C++ в частности и программированию в целом! Сейчас же давайте закрепим пройденный материал.

Оглавление:

1. **Теория**
2. **Тест**

Теория

Блок стейтментов (или ***«составной оператор»***) обрабатывается компилятором так, как если бы это был один стейтмент. Составные операторы помещаются в фигурные скобки `{` и `}` и используются почти везде.

Локальные переменные создаются в точке определения и уничтожаются при выходе из блока, в котором они объявлены. Доступ к ним возможен только внутри этого же блока.

Глобальные переменные создаются, когда программа запускается, и уничтожаются, когда она завершает свое выполнение. Они могут использоваться в любом месте программы. Неконстантные глобальные переменные следует избегать, потому что это — зло.

Ключевое слово static может использоваться для преобразования глобальной переменной во внутреннюю (с внутренней связью), чтобы её можно было использовать только в том файле, в котором она объявлена. Также ключевое слово static используют для указания того, что локальная переменная должна иметь статическую продолжительность жизни. А это означает, что она будет сохранять свое значение даже после выхода из своей области видимости.

Пространство имен — это область, в которой гарантируется уникальность всех имен. Отличный способ избежать конфликтов имен. Не используйте using-стейтменты вне тела функций.

Неявное преобразование типов данных происходит, когда один тип данных конвертируется в другой тип без использования одного из операторов явного преобразования. **Явное преобразование типа** происходит, когда один тип данных конвертируется в другой с помощью одного из операторов явного преобразования. В некоторых случаях это абсолютно безопасно, в других случаях данные могут быть потеряны. Избегайте использования конвертации C-style, вместо нее используйте оператор static_cast.

std::string — это простой способ работы с текстовыми строками (текст помещается в двойные кавычки).

Перечисления позволяют создавать собственные (пользовательские) типы данных. **Классы enum** — это те же перечисления, но надежнее и безопаснее. Используйте их вместо обычных перечислений, если ваш компилятор поддерживает C++11.

typedef позволяет создавать псевдонимы для типов данных. Целочисленные типы данных с фиксированным размером реализованы с помощью typedef. Псевдонимы типов полезны для присваивания простых имен сложным типам данных.

И, наконец, **структуры**. Они позволяют сгруппировать отдельные переменные в единое целое. Доступ к членам структуры осуществляется через оператор выбора членов (`.`). Объектно-ориентированное программирование в значительной степени основывается именно на структурах, поэтому, если вы изучили только одну вещь из этой главы, то лучше, чтобы это были структуры.

Тест

При разработке игры мы решили, что в ней должны быть монстры, потому что всем нравится сражаться с монстрами. Объявите структуру, которая представляет вашего монстра. Монстр может быть разным: `ogre`, `goblin`, `skeleton`, `orc` и `troll`. Если ваш компилятор поддерживает C++11, то используйте классы enum, если нет — обычные перечисления.

Каждый монстр также должен иметь имя (используйте std::string) и количество здоровья, которое отображает, сколько урона он может получить, прежде чем умрет. Напишите функцию printMonster(), которая выведет все члены структуры. Объявите монстров типа `goblin` и `orc`, инициализируйте их, используя список инициализаторов, и передайте в функцию printMonster().

Пример результата выполнения вашей программы:

```
This Goblin is named John and has 170 health.
This Orc is named James and has 35 health.
```

Ответ C++11

```cpp
#include <iostream>
#include <string>

// Определяем класс enum с типами монстров
enum class MonsterType
{
    OGRE,
    GOBLIN,
    SKELETON,
    ORC,
    TROLL
};

// Наша структура представляет одного монстра
struct Monster
{
    MonsterType type;
    std::string name;
    int health;
};

// Возвращаем тип монстра в виде строки
std::string getMonsterTypeString(Monster monster)
{
    if (monster.type == MonsterType::OGRE)
        return "Ogre";
    if (monster.type == MonsterType::GOBLIN)
        return "Goblin";
    if (monster.type == MonsterType::SKELETON)
        return "Skeleton";
    if (monster.type == MonsterType::ORC)
        return "Orc";
    if (monster.type == MonsterType::TROLL)
        return "Troll";

    return "Unknown";
}

// Выводим информацию о монстре
void printMonster(Monster monster)
{
    std::cout << "This " << getMonsterTypeString(monster);
    std::cout << " is named " << monster.name << " and has " << monster.health << " health.\n";
}

int main()
{
    Monster goblin = { MonsterType::GOBLIN, "John", 170 };
    Monster orc = { MonsterType::ORC, "James", 35 };

    printMonster(goblin);
    printMonster(orc);

    return 0;
}
```

Ответ до C++11

```cpp
#include <iostream>
#include <string>

// Определяем перечисление с типами монстров
enum MonsterType
{
    MONSTER_OGRE,
    MONSTER_GOBLIN,
    MONSTER_SKELETON,
    MONSTER_ORC,
    MONSTER_TROLL
};

// Наша структура представляет одного монстра
struct Monster
{
    MonsterType type;
    std::string name;
    int health;
};

// Возвращаем тип монстра в виде строки
std::string getMonsterTypeString(Monster monster)
{
    if (monster.type == MONSTER_OGRE)
        return "Ogre";
    if (monster.type == MONSTER_GOBLIN)
        return "Goblin";
    if (monster.type == MONSTER_SKELETON)
        return "Skeleton";
    if (monster.type == MONSTER_ORC)
        return "Orc";
    if (monster.type == MONSTER_TROLL)
        return "Troll";

    return "Unknown";
}

// Выводим информацию о монстре
void printMonster(Monster monster)
{
    std::cout << "This " << getMonsterTypeString(monster);
    std::cout << " is named " << monster.name << " and has " << monster.health << " health.\n";
}

int main()
{
    Monster goblin = { MONSTER_GOBLIN, "John", 170};
    Monster orc = { MONSTER_ORC, "James", 35};

    printMonster(goblin);
    printMonster(orc);

    return 0;
}
```